

Quantum Virtual Machine (QVM) Runtime: **Write Once, Run Anywhere** (WORA) for Multiple Backends

A Final Year Project dedicated to achieving hardware-agnostic quantum program execution, abstracting away vendor-specific complexity.

Project Overview: The WORA Philosophy for Quantum

This project defines the design and development of a Quantum Virtual Machine (QVM) or runtime system, analogous to classical virtual machines, but focused on portability in quantum computing.



Hardware Agnostic

Abstracts hardware-specific differences such as gate sets, noise models, and connectivity requirements.



Seamless Portability

Enables quantum algorithms written once (e.g., using Qiskit or Cirq) to run seamlessly on multiple target backends.

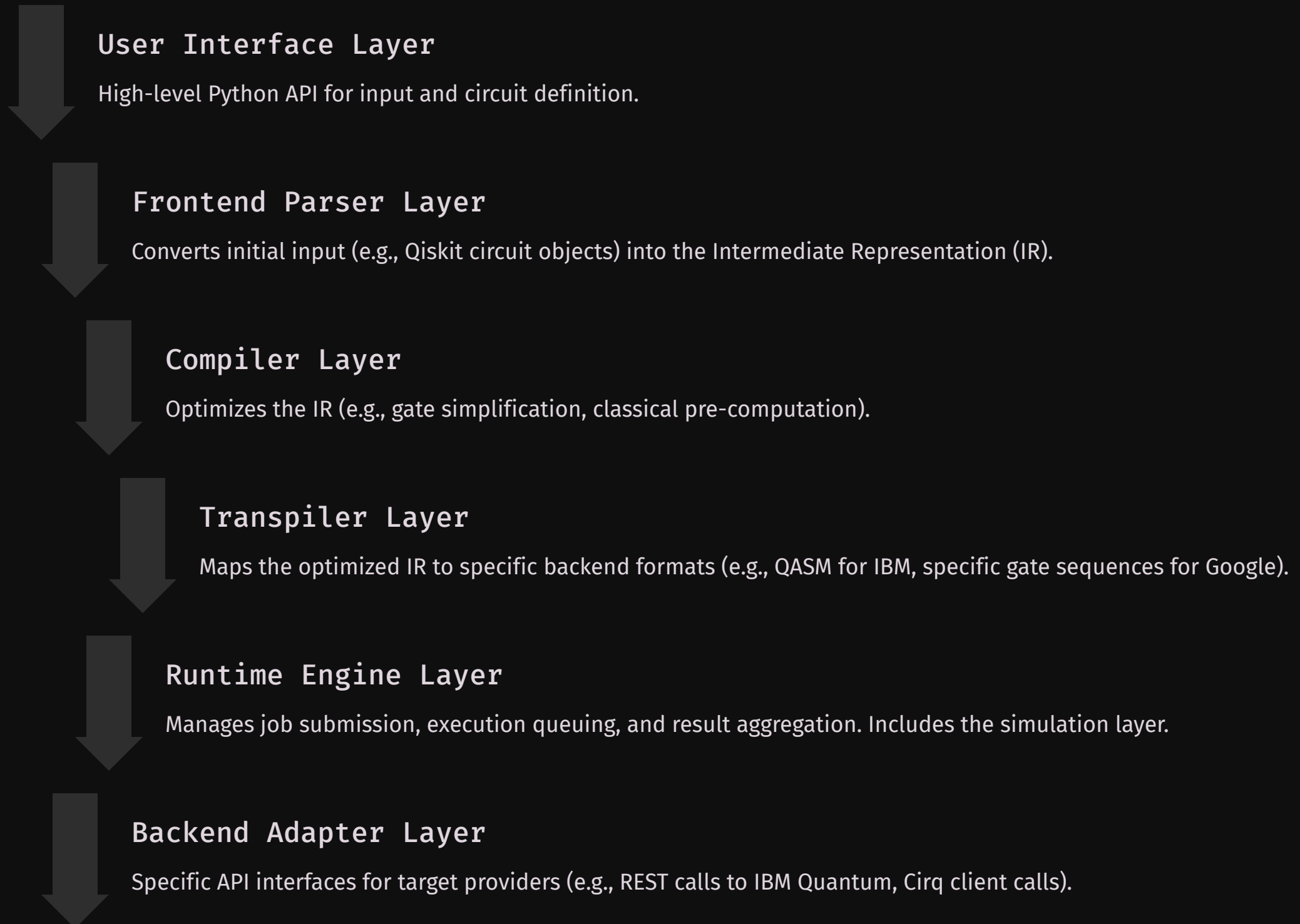


Research Prototype

A proof-of-concept system focusing on simple algorithms (Bell states, Grover's search) to demonstrate viable cross-platform execution.

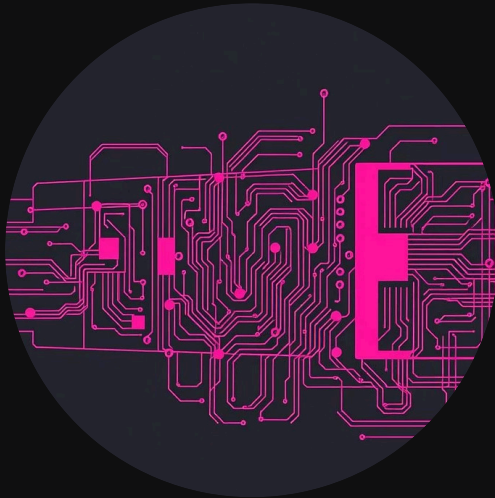
Architectural Layers: Modular Design for Extensibility

The QVM employs a layered architecture for modularity, enabling easy maintenance and the straightforward addition of new quantum providers.



Core Product & Key Deliverables

The QVM runtime is built on an Intermediate Representation (IR) to ensure hardware agnosticism, supported by custom transpilers and a robust execution engine.



Modular QVM Runtime

Accepts quantum programs, transpiles them via an IR, and executes them on target backends (Qiskit/Cirq initial support).



Hardware-Agnostic IR

A key component ensuring portability by decoupling the high-level program from the low-level hardware constraints.



Extensible Backend Adapters

Allows for easy addition of new quantum providers (Azure Quantum, Rigetti) without rewriting core logic.

Goal: Demonstrate viable WORA for quantum, with success metrics focused on fidelity and reduced transpilation errors across backends.

Phase 1 & 2: Foundations and Prototyping

The initial two phases focus heavily on research, defining the requirements, and implementing the fundamental architectural components of the QVM.

Phase 1: Research and Planning (Weeks 1-4)

- Analyze existing frameworks (PennyLane, pytket, QIR).
- Define core requirements and target layers.
- Establish success metric: >80% fidelity across backends.
- Setup development environment (Python, Qiskit, Cirq, GitHub).

Phase 2: Prototyping Core Components (Weeks 5-12)

- Build Frontend Parser for Qiskit/Cirq input.
- Design and implement the Intermediate Representation (IR).
- Implement the basic Transpiler for conversion to the first backend (e.g., IBM).

Phase 3 & 4: Execution and Expansion

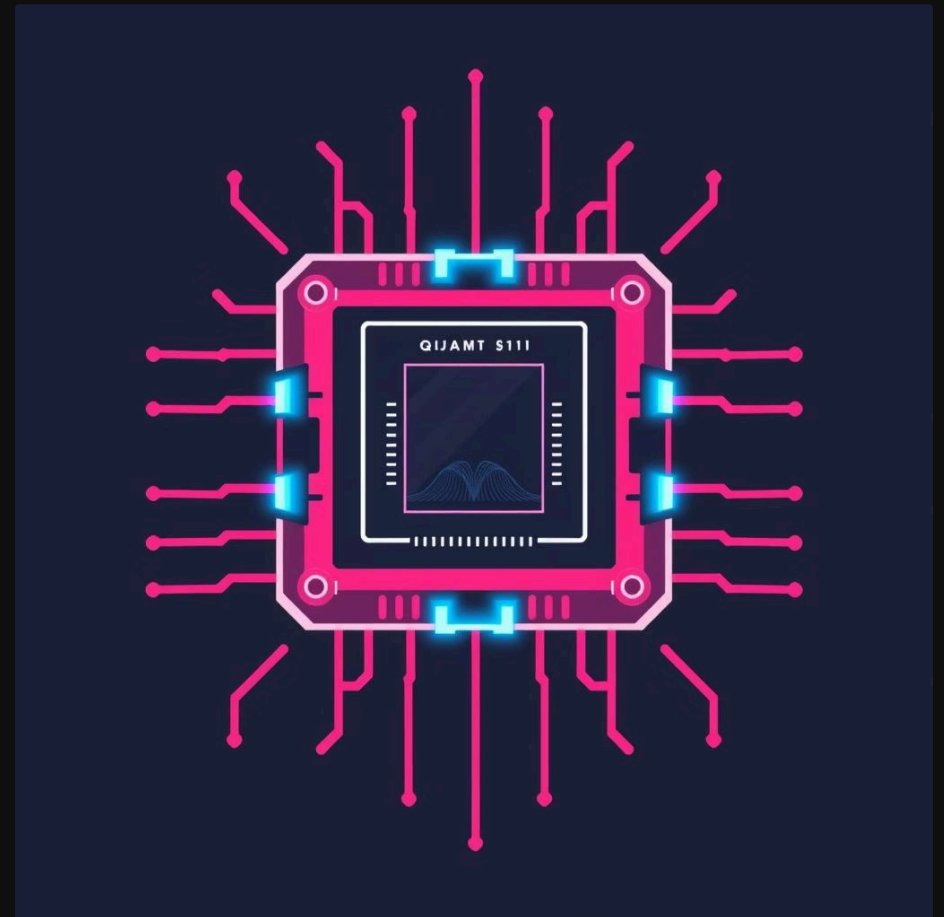
Following the core design, we shift focus to operational functionality: executing jobs, testing on real quantum hardware, and expanding the supported backend ecosystem.

Phase 3: Runtime and Execution (Weeks 13-20)

- Develop Runtime Engine for job management.
- Integrate simulation layer (e.g., custom simulator or Cirq QVM).
- Test execution with real backends via free tiers.

Phase 4: Expansion and Optimization (Weeks 21-28)

- Add support for a second distinct backend (e.g., Google Cirq).
- Implement performance optimizations like noise mitigation.
- Conduct initial user testing and gather feedback.



Hurdles and Mitigation Strategies

Building a hardware-agnostic quantum runtime presents significant technical and resource-based challenges that require proactive mitigation.

Technical: Quantum Noise and Errors

Backends have varying fidelities. Mitigation requires advanced algorithms and smart transpilation to reduce error susceptibility.

Technical: Gate Incompatibilities

Different providers use different native gate sets (e.g., IBM CNOT vs. Google CZ). The Transpiler must provide intelligent, low-error mappings.

Resource: Hardware Access Limits

Free hardware tiers impose queueing and usage restrictions, limiting large-scale testing.

- Mitigation: Start testing primarily with reliable simulators.

Resource: Time Constraints (FYP)

Limited timeline requires strict feature prioritization. The focus remains on core WORA functionality, not exhaustive features.

Key Development Resources and Tools

The project leverages established industry tools and open-source quantum frameworks for rapid development and inspiration.



Frameworks & Libraries

- **Qiskit (IBM):** Circuit building and IBM backend integration.
- **Cirq (Google):** Google backend connectivity and integrated simulation.
- **PennyLane/pytket:** Key references for existing quantum portability solutions.
- **QIR (Microsoft):** Reference for designing the hardware-agnostic Intermediate Representation.

Development & Learning Tools

- **Python/Rust:** Primary implementation languages.
- **GitHub:** Version control, collaboration, and open-source community engagement.
- **Jupyter:** Interactive environment for prototyping and demo scripts.
- **Nielsen & Chuang:** Foundational knowledge reference.

Testing and Validation Strategy

Rigorous testing at multiple levels ensures the QVM runtime is both correct (unit tests) and performs reliably across disparate hardware platforms (integration and performance testing).



Unit Testing

Using pytest to verify correctness of core components (e.g., parser accuracy, gate mapping in the transpiler).



Integration Testing

End-to-end circuit execution: write once, transpile, run on 2+ backends, and verify result coherence.



Performance Benchmarking

Measuring fidelity, execution runtime, and error rates across different backends for comparison.

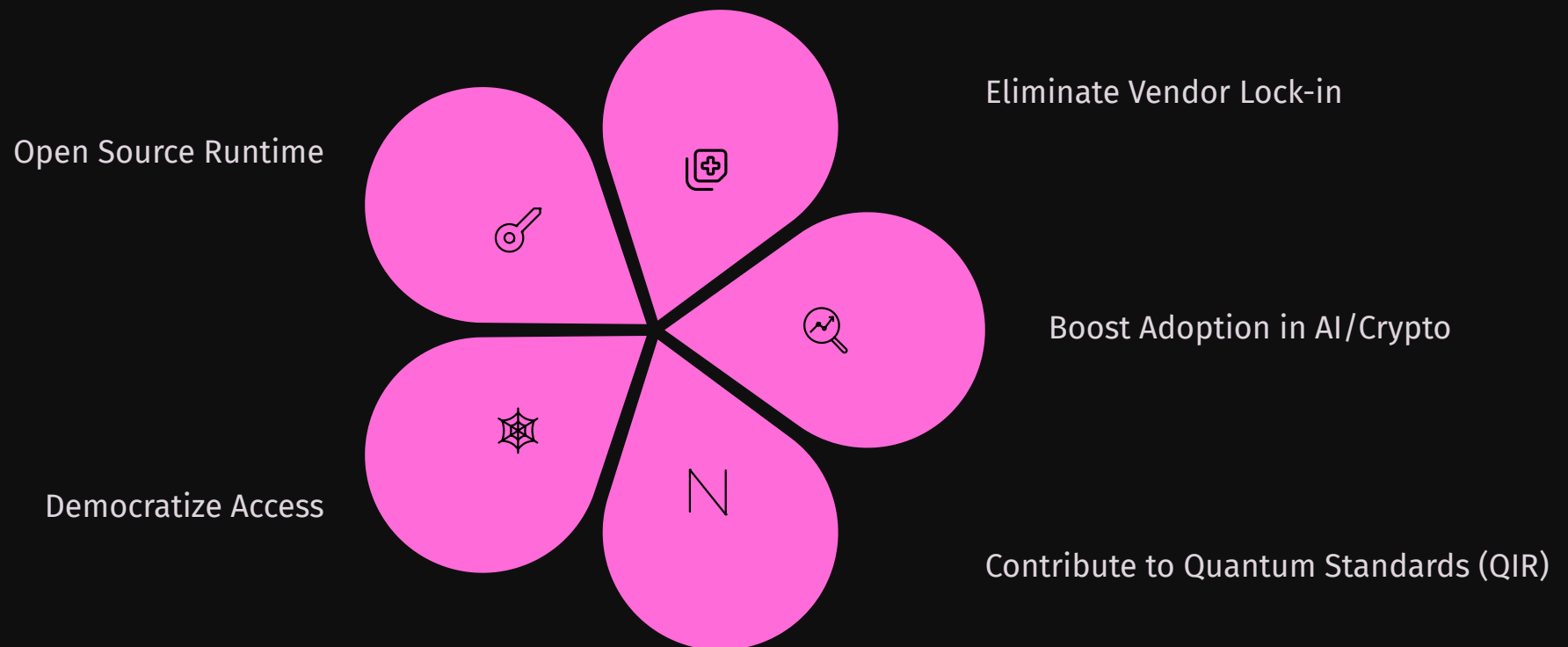


Success Metrics

Target: Circuits run on 2+ backends with **<10% fidelity loss**, verifiable using Qiskit's fidelity calculators.

Ultimate Goal: Democratizing Quantum Development

The QVM aims to eliminate vendor lock-in, enabling broader adoption and contribution to the evolving field of quantum computing standards.



Demonstrate viable WORA for quantum with measurable outcomes, setting the stage for future commercial or academic tools.